

Objective

The goal of this exercise is to design and implement a C++ library for processing one-dimensional time series. The library must rely on templates, inheritance, and polymorphism, and expose its functionality to Python using pybind11.

Background

A time series is an ordered sequence of numerical samples

$$S = \{s_1, s_2, \dots, s_n\}.$$

The following operations are defined.

- **Scaling** by a factor α : $s'_i = \alpha s_i$.
- **Normalization**: $s'_i = \frac{s_i - \min(S)}{\max(S) - \min(S)}$, assuming $\max(S) \neq \min(S)$.
- **Moving average** with window size k : $s'_i = \frac{\sum_{j=i}^{i+k-1} s_j}{k}$, assuming $k \neq 0$.

The arithmetic mean of a non-empty time series is $\mu = \frac{\sum_{i=1}^n s_i}{n}$.

Instructions

Implement a reusable C++ library for processing time series.

Tasks

1. (2 points) Define a templated `TimeSeries` class

Define a templated class `TimeSeries<T>` that:

- Stores the samples.
- Provides suitable constructors.
- Provides a `size()` member function.

2. (2 points) Sample access

- Implement `operator[]` for reading from and writing to a specific sample.
- Ensure const-correctness.
- Accessing an invalid index must throw an exception.
- Follow RAII principles and ensure memory safety.

3. (3 points) Time series transformations

Define the abstract base class

```
template<typename T>
class TimeSeriesTransform {
public:
    virtual TimeSeries<T> apply(const TimeSeries<T>& s) const = 0;
    // ...
};
```

Implement the following derived classes:

- `ScaleOp<T>`;
- `NormalizeOp<T>`;

- `MovingAverageOp<T>`.

Each derived class must:

- Override `apply()`.
- Receive its parameters through the constructor.
- Validate the input parameters.
- Throw exceptions whenever appropriate.

4. (2 points) Mean computation

Implement a `MeanOp<T>` class that computes the arithmetic mean of a time series.

- Clearly define how the scalar result is returned.
- Empty time series must generate an exception.
- The implementation should avoid unnecessary copies.

5. (1 point) Testing and polymorphism

In `main()`:

- Demonstrate runtime polymorphism by applying different transformations through a base-class pointer or reference.
- Verify that the implementation works for at least two numeric types (e.g., `int` and `double`).

6. (1 point) Configuration and compilation

- Write a `CMakeLists.txt` file to build the library and a test executable.
- Enable the C++17 standard.
- Provide concise build instructions.

7. (4 points) Python bindings using `pybind11`

- Expose `TimeSeries` and all transformation classes to Python.
- Ensure that C++ exceptions are correctly translated into Python exceptions.
- Write a Python script that:
 - Creates a time series.
 - Applies several transformations.
 - Computes the arithmetic mean.
 - Compares the execution time with equivalent NumPy implementations on large datasets.
 - Briefly comments on the observed results.

Evaluation criteria

- Correct use of templates, inheritance, and polymorphism.
- Correct implementation of all transformations.
- Proper handling of edge cases and exceptions.
- Memory safety and RAII compliance.
- Clean and usable Python bindings.
- Meaningful comparison with NumPy.
- Quality and clarity of the CMake configuration.
- Code organization and readability.